Semantic tagging

AI Integration — Local LLM Server · built Nov 13, 2026

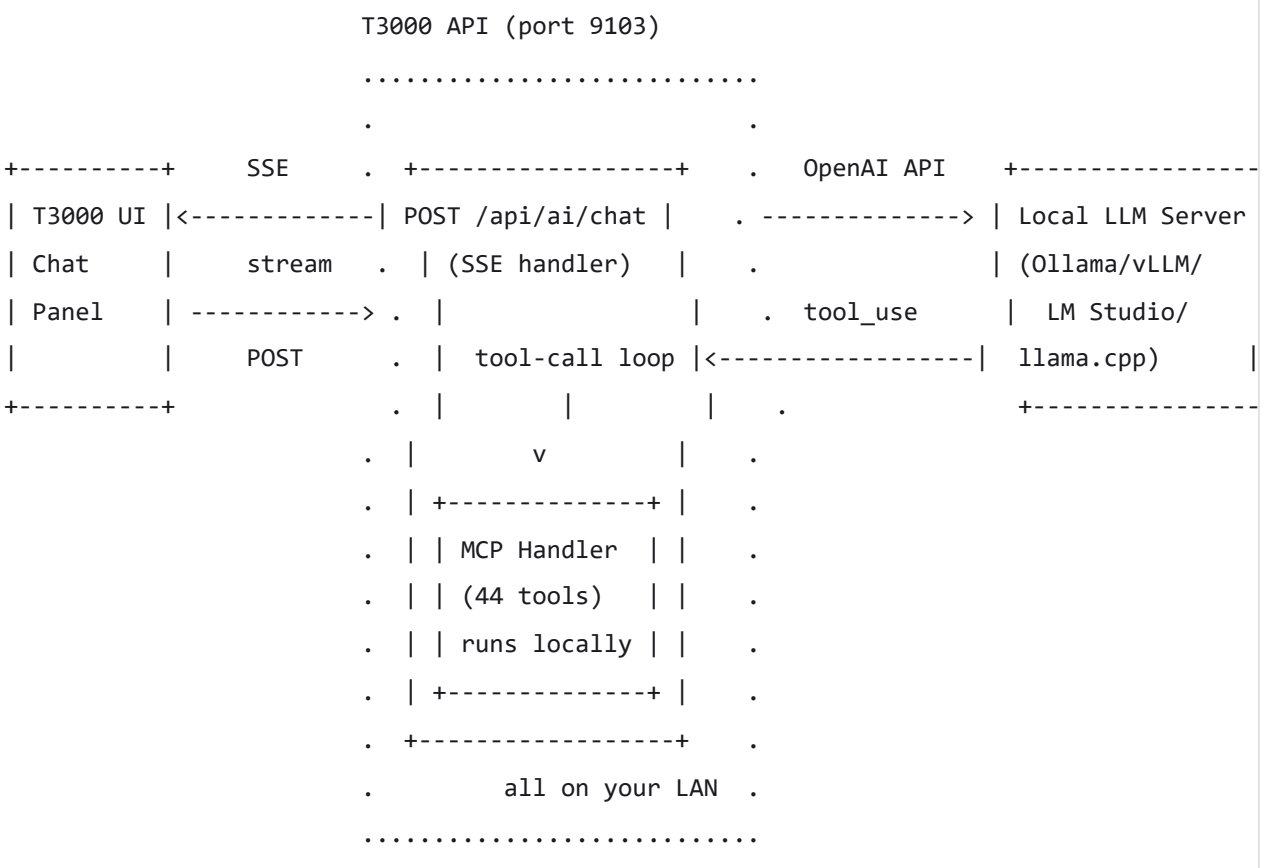
AI Integration — Local LLM Server

⬅️ Back to Docs | Cloud LLM APIs | AI Chat Page Design

How to run a local LLM server on your network and connect T3000 to it. No cloud API keys needed — all processing stays inside your building.

Architecture

TEXT



1. Browser sends user message to `POST /api/ai/chat`
2. T3000 forwards to your local LLM via OpenAI-compatible `/v1/chat/completions`
3. LLM streams response — text goes straight to browser via SSE

4. When LLM wants to call a tool, T3000 intercepts the `tool_use` event
5. Tool is executed **locally** by the MCP handler — never sent to the LLM
6. Tool result is sent back to the LLM, which continues the stream

No building data leaves your network. No cloud API keys needed.

Option 1: Ollama (easiest)

Install

Windows / Mac / Linux: Download from ollama.com

Or on Linux:

```shell
curl -fsSL https://ollama.com/install.sh | sh
```

Pull a Model

```shell
## Best for tool calling
ollama pull llama3.1:8b
ollama pull qwen2.5:7b
ollama pull mistral:7b

## Smaller/faster
ollama pull phi3:mini
ollama pull gemma2:2b
```

Configure for Tool Calling

Ollama needs a Modelfile to support OpenAI-compatible tool calling:

```shell
ollama pull llama3.1:8b

## Create a custom modelfile (optional but helps with structured outputs)
cat > Modelfile << 'EOF'
FROM llama3.1:8b
PARAMETER temperature 0.1
PARAMETER num_ctx 8192
SYSTEM You are a building automation assistant. You have access to tools that query devi
EOF

ollama create t3k-assistant -f Modelfile
```

Verify It's Running

```shell
ollama serve
## or just run:
ollama list
```

Default port: `11434`

T3000 Settings

Field	Value
Provider	Local
Model	`llama3.1:8b` or your model name
Endpoint	`http://localhost:11434/v1`
API Key	(leave blank — Ollama has no auth by default)

If T3000 is on a different machine, use the server's LAN IP:

```text
http://192.168.1.50:11434/v1
```

Tool Calling Support

Model	Tool calling	Notes
`llama3.1:8b`	✅ Good	Best overall for tool use
`qwen2.5:7b`	✅ Good	Strong at structured outputs
`mistral:7b`	⚠️ Partial	May miss tools occasionally
`phi3:mini`	❌ Limited	Not recommended for tool chains

Option 2: vLLM (production-grade)

Install

```shell
pip install vllm
```

Start Server with Tool-Ready Model

```shell
python -m vllm.entrypoints.openai.api_server \
  --model Qwen/Qwen2.5-7B-Instruct \
  --port 8000 \
  --served-model-name qwen2.5 \
  --enable-auto-tool-choice \
  --tool-call-parser hermes
```

For GPU:

```shell
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3.1-8B-Instruct \
  --port 8000 \
  --tensor-parallel-size 2 \
  --enable-auto-tool-choice \
  --tool-call-parser llama3_json
```

T3000 Settings

Field	Value
Provider	Local
Model	`Llama-3.1-8B-Instruct`
Endpoint	`http://localhost:8000/v1`
API Key	`token vllm` (default vLLM auth)

Hardware Requirements

Model Size	VRAM	RAM (CPU-only)
7B (qwen2.5)	8 GB	16 GB
8B (llama3.1)	10 GB	20 GB
70B (llama3.1)	40 GB × 2	Not recommended

Option 3: LM Studio (GUI, easiest setup)

Install

1. Download from lmstudio.ai
2. Install and open the app
3. Search for `Llama 3.1 8B Instruct` or `Qwen 2.5 7B Instruct`
4. Download the model
5. Go to **Developer** tab → **Start Server**
6. Set port to `1234`

T3000 Settings

Field	Value
Provider	Local
Model	`llama-3.1-8b-instruct`
Endpoint	`http://localhost:1234/v1`
API Key	(leave blank)

Option 4: llama.cpp (lightweight, no GPU needed)

Install

SHELL

```
## Clone and build
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp
make -j4

## Download a GGUF model (example: Qwen 2.5 7B Q4)
wget https://huggingface.co/bartowski/Qwen2.5-7B-Instruct-GGUF/resolve/main/Qwen2.5-7B-I
```

Start Server

SHELL

```
./llama-server \
  -m Qwen2.5-7B-Instruct-Q4_K_M.gguf \
  --host 0.0.0.0 \
  --port 8080 \
  -c 8192
```

T3000 Settings

Field	Value
Provider	Local
Model	`qwen2.5-7b`
Endpoint	`http://localhost:8080/v1`
API Key	(leave blank)

Network Security for Local Servers

If T3000 and LLM are on the SAME machine

TEXT

```
Endpoint: http://localhost:{port}/v1
```

No network exposure — safest option.

If T3000 and LLM are on DIFFERENT machines (same LAN)

```text
Endpoint: http://192.168.1.50:{port}/v1
```

For Ollama, you must also allow LAN access:

```shell
## On the Ollama server machine:
set OLLAMA_HOST=0.0.0.0
ollama serve
```

Firewall: Ensure port `11434` (Ollama), `8000` (vLLM), or your custom port is open on the LLM machine's firewall.

Do NOT expose to the internet

Local LLM servers have no authentication by default. Only use on your internal network behind a firewall. For remote access, use a VPN.

API Format (OpenAI-Compatible)

All local servers use the same OpenAI-compatible chat completions API:

```text
POST http://{host}:{port}/v1/chat/completions
```

Request:

JSON

```
{
  "model": "llama3.1:8b",
  "messages": [
    {"role": "system", "content": "You are a building automation assistant..."},
    {"role": "user", "content": "What's AHU-1 supply temperature?"}
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "t3000_haystack_search_points",
        "description": "Search points by Haystack tags.",
        "parameters": {
          "type": "object",
          "properties": {
            "tags": {"type": "array", "items": {"type": "string"}},
            "serial_numbers": {"type": "array", "items": {"type": "integer"}}
          },
          "required": ["tags"]
        }
      }
    },
    {
      "type": "function",
      "function": {
        "name": "t3000_point_read",
        "description": "Read live value of a point.",
        "parameters": {
          "type": "object",
          "properties": {
            "serial_number": {"type": "integer"},
            "point_type": {"type": "string", "enum": ["INPUT", "OUTPUT", "VARIABLE"]},
            "point_index": {"type": "integer"}
          },
          "required": ["serial_number", "point_type", "point_index"]
        }
      }
    }
  ],
```

```
  "stream": true
}
```

Streaming response: SSE chunks with `data: {"choices":[{"delta":{"content":"The "}}]}` format.

Tool call response:

JSON

```
{
  "choices": [
    {
      "delta": {
        "tool_calls": [
          {
            "index": 0,
            "id": "call_abc123",
            "function": {
              "name": "t3000_haystack_search_points",
              "arguments": "{\"tags\":[\"supply\",\"air\",\"temp\"]}"
            }
          }
        ]
      }
    }
  ]
}
```

Tool result format (send back):

JSON

```
{
  "role": "tool",
  "tool_call_id": "call_abc123",
  "content": "[{\"label\":\"AHU-1 Supply Temp\",\"value\":55.2}]"
}
```

Recommended Local Models

Model	Size	Tool Calling	Notes
`llama3.1:8b`	8B	✅ Excellent	Best all-around for tool use
`qwen2.5:7b`	7B	✅ Excellent	Strong at JSON/formatted output
`mistral-nemo:12b`	12B	✅ Good	Larger context window
`gemma2:9b`	9B	⚠️ Good	May need prompt tuning
`phi3:mini`	3.8B	❌ Weak	Too small for reliable tool calls

Troubleshooting

"Model does not support tools"

Your model is too old or doesn't support function calling. Switch to `llama3.1:8b` or `qwen2.5:7b`.

"Connection refused"

The LLM server isn't running. Check:

```shell
curl http://localhost:11434/api/tags      # OLlama
curl http://localhost:8000/v1/models      # vLLM
```

"Tool call returned empty"

The model didn't understand the prompt. Try:

- More explicit tool descriptions
- Adding examples in the system prompt
- Using a larger model (8B+ instead of 3B)

"Response is slow"

- Check CPU/GPU usage on the LLM server
- Reduce `num_ctx` (context length) for faster inference
- Use a smaller model for simple queries, larger for complex ones